Base Radio Controller for TX (ExpressLRS) – 2400 MHz

OR STM MCU

SCREEN MCU

Controller_Screen_DC
Controller_Screen_MOSI_SDA
Controller_Screen_SCK
Controller_Screen_CS

ADCVBAT

Pins:
0 – SW
1 – SW
10 – SW
9 – SW

A7 – VBAT_ADC

5 – LED
6 – LED

A2 – Joystick 1
A3 – Joystick 1
4 – Joystick 1

A1 – Joystick 2
A0 – Joystick 2
2 – Joystick 2

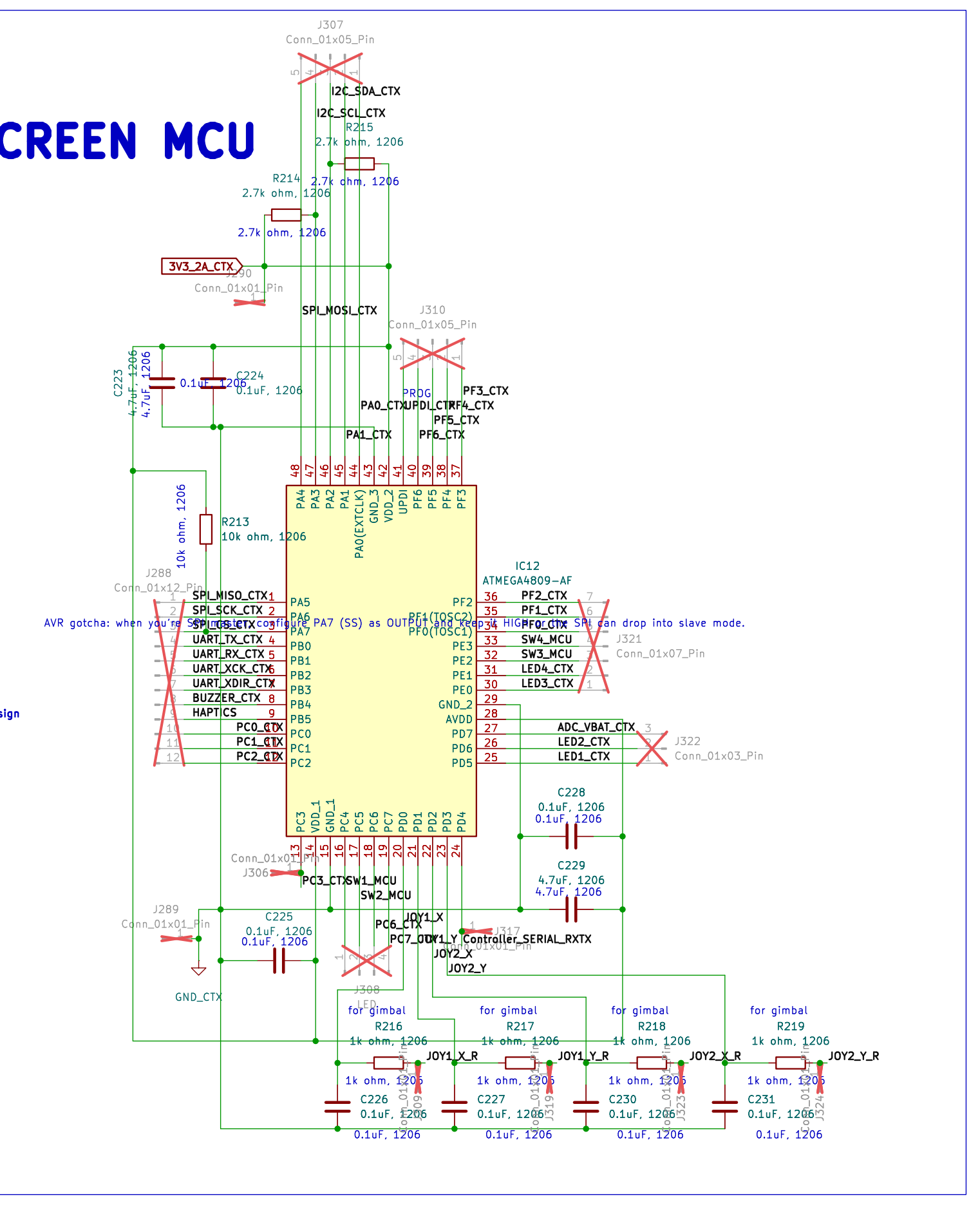
SPI/UART – Controller TX & Serial Wire Programming

ICSP (programming pins for IC (SPI))

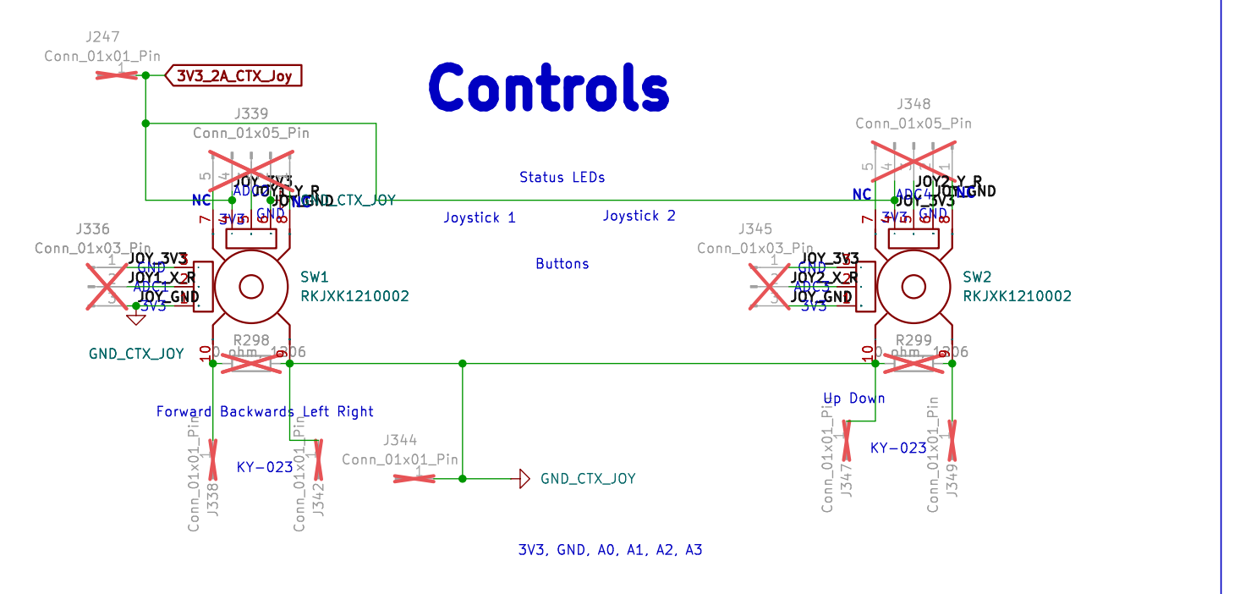
ATMEGA (ARDUINO) MCU (pro mini) – Check example design

ATmega328P

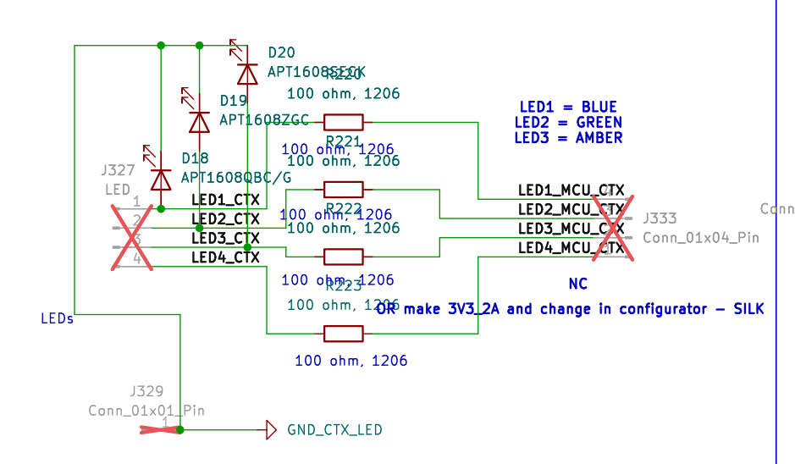
ATmega4809



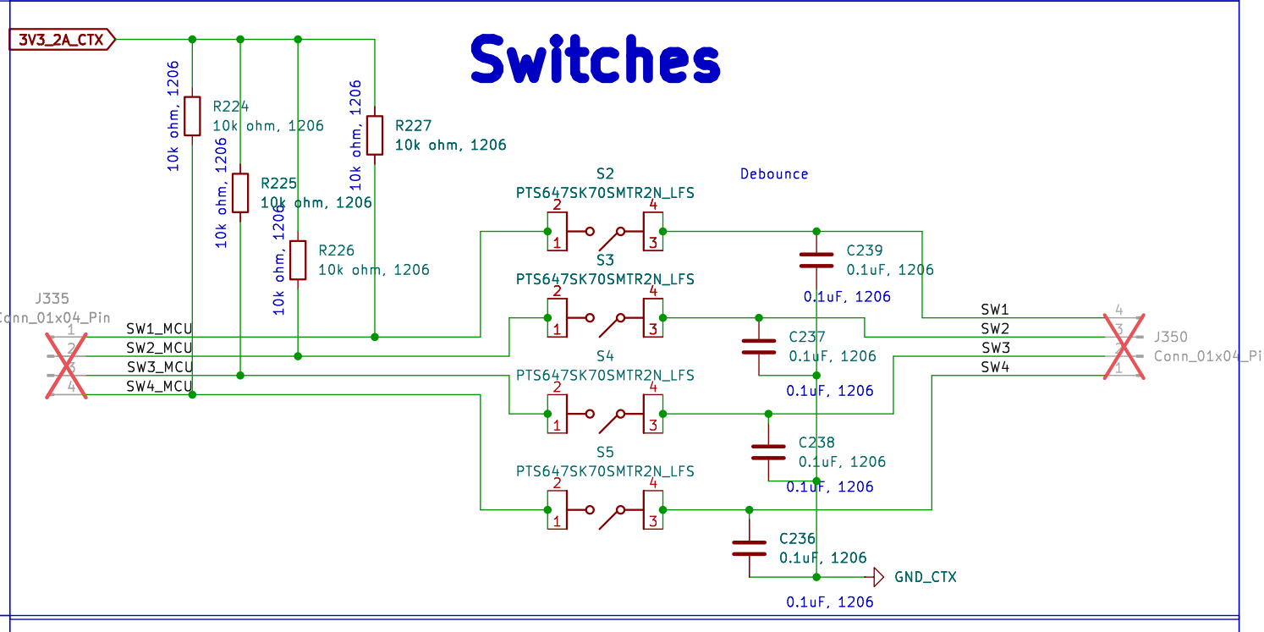
Controls



LEDs



Switches



BIG CONTROLLER

Matek M8Q–5883 GPS

https://www.rcgroups.com/forums/showthread.php?3869543–Blog–17–RadioMaster–TX16S–schematic–diagram

for TX, you generally have a different chip as the main uC, that communicates with the esp radio via uart..

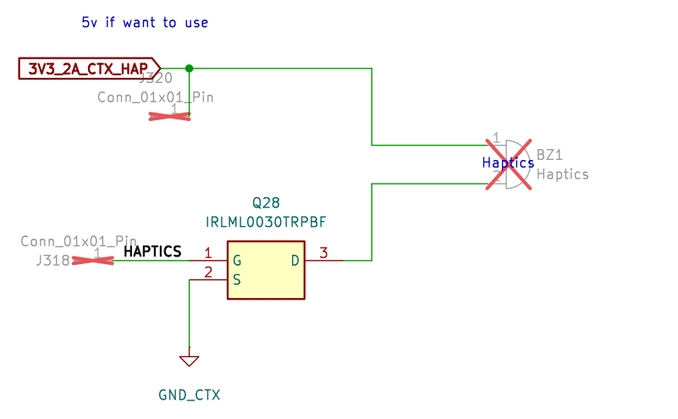
on a RX, the esp32 is used as the main uC

EdgeTX compatible radio

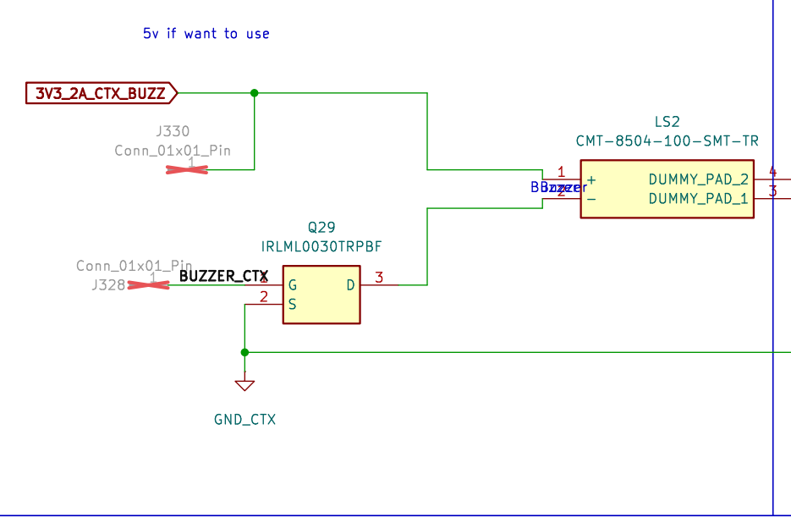
LITTLE CONTROLLER

https://github.com/TheDIYGuy999/RC_Transmitter

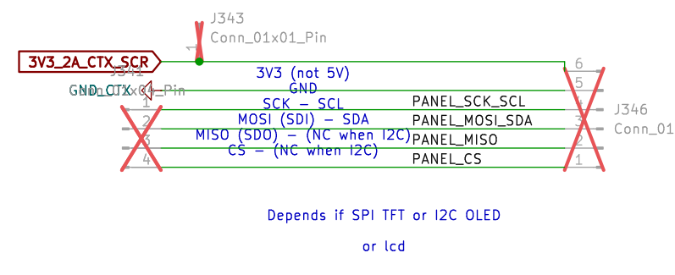
Haptics



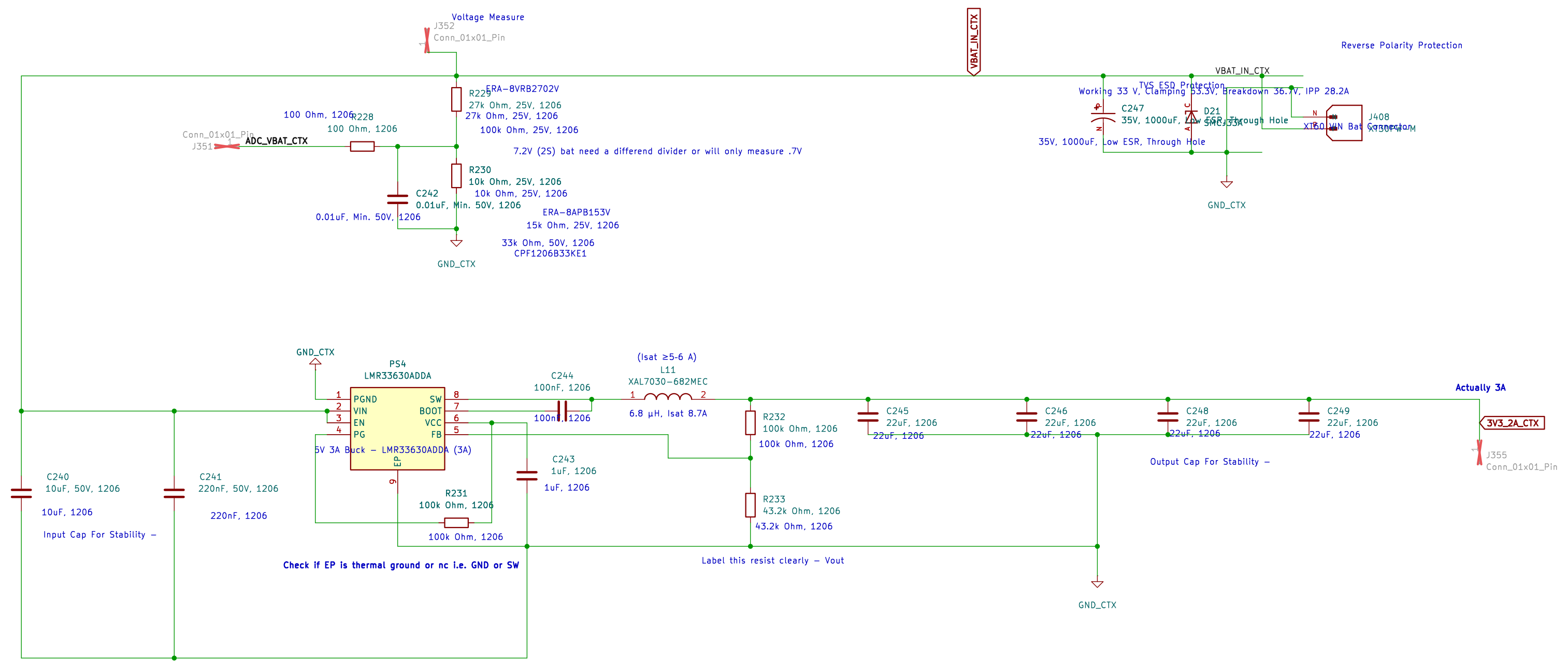
BUZZER



Screen Connection



Power CTRLR



Future design all on one IC

stm32 cannot be used anymore, since support for it got removed
you could use it with old versions of the code, but not with current code

SX1280IMLTRT for rx and same + amp for tx from gnd

Controller TX (ExpressLRS) – 2400 MHz

900MHz Available option

ESP–01F

stm32l432kbu6

esp32

Fiducials all work!

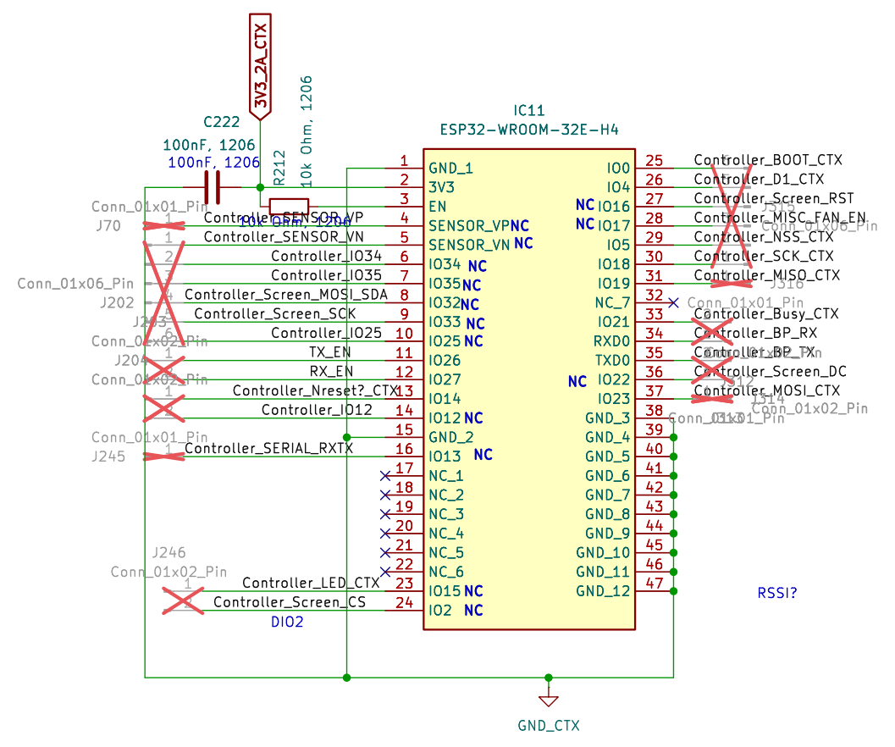
Fiducials all work!

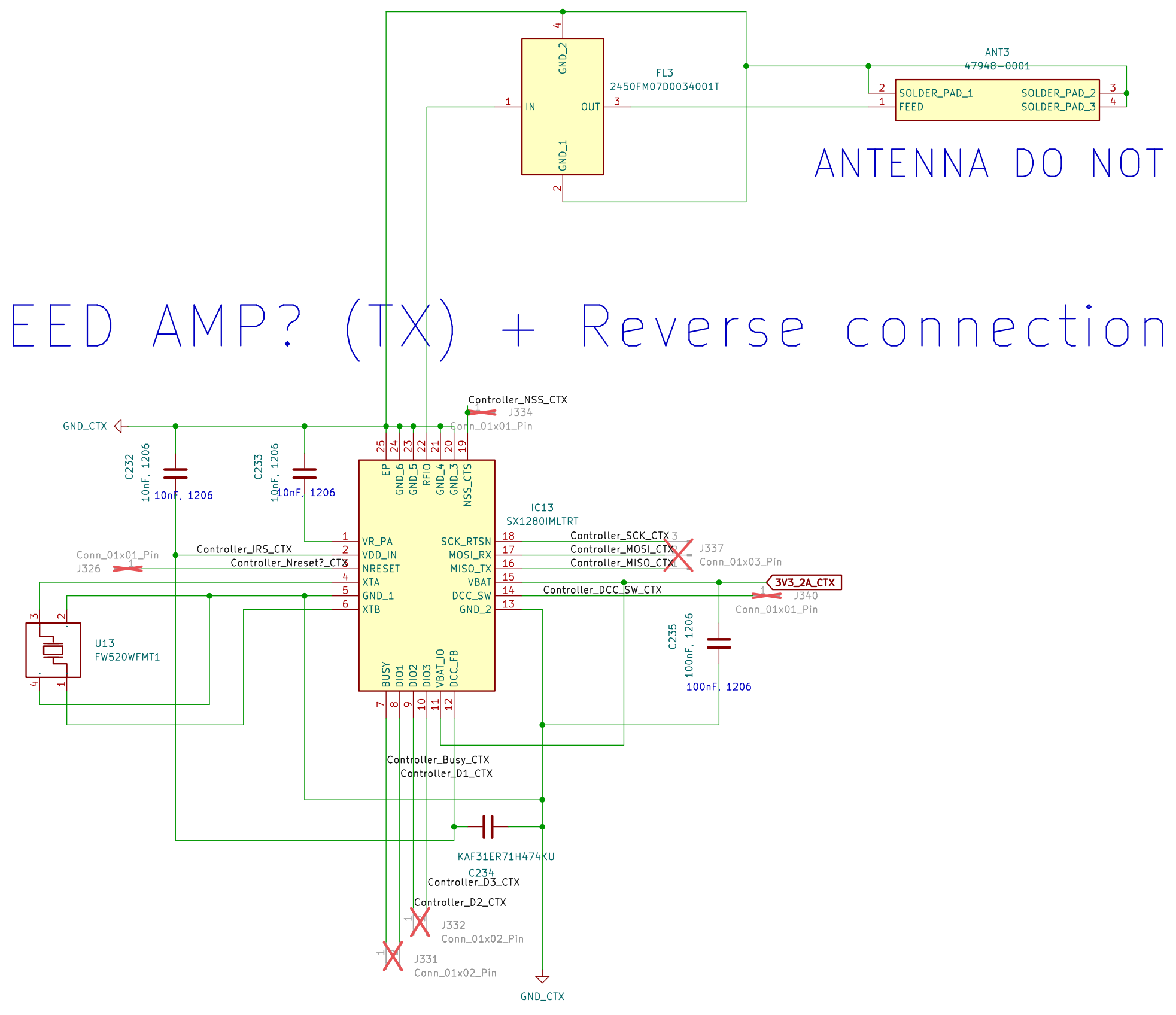
CHECK DATASHEET FOR PARTS PROGRAMMING

https://github.com/ExpressLRS/ExpressLRS–Hardware/tree/master/PCB/2400MHz/TX_SX1280_Minimal

ANTENNA DO NOT FORGET

NEED AMP? (TX) + Reverse connection? OR RFIO GOES BOTH WAYS





If doesnt work E28 target, try rewire for https://github.com/ExpressLRS/targets/blob/master/TX/DIY%202400%20Mini.json

https://github.com/ExpressLRS/targets/blob/master/TX/DIY%202400%20E28.json

for TX, you generally have a different chip as the main uC, that communicates with the esp radio via uart..

on a RX, the esp32 is used as the main uC

EdgeTX compatible radio

PPM Pin (JR Bay Example)
JR Pin Function
Pin 1 GND
Pin 2 +V (8-12 V)
Pin 3 TX (telemetry back to radio)
Pin 4 RX (stick data to module for CRSF)
Pin 5 PPM input (legacy)
Pin 6 Module detect

a) CRSF Protocol (Crossfire Serial)

Developed by Team BlackSheep (TBS), but adopted by ELRS.

A bi–directional UART serial link running at 400,000 baud (400 kbaud).

from my experience
standard via drill size >= 0.3/0.4mm
less layer count, 4 is the sweet spot
stick to common and easy–to–solder packaging (qfp/qfn, 0402 min, no bga)
surface finish, hasl leaded is the cheapest in most fab
panelize boards for higher quantity
single sided assembly
optimize bom – consolidate diff component values (i.e. 2x 1k when 1k is already used elsewhere vs. 2k and 1k)
jlc specific
maximize their basic parts library in ur bom
use their economic assembly
Osiris

– Yesterday at 17:28
But that could result in less well designed PCBs.
I personally wouldn't fly a 4 layer FC
That most likely would result in frequently corner cutting with regard to PCB best practices
emrick – Yesterday at 17:29
for 6 layers, is S–G–S–P–G–S good for FC?
Osiris

– Yesterday at 17:29
(unless the FC is huge)
Osiris

– Yesterday at 17:30
you could use the power plane as return path yes

Core assumptions

Power: 2S → LMR33630 → 3V3 rail.

VBAT sense: 27 k (top) / 10 k (bottom) to ADC with 100 nF at the MCU pin.

Output to ELRS (now): PPM.

Haptic motor and magnetic buzzer: low–side MOSFET (IRLML0030), SS14 flyback across load (cathode to +3V3, anode to switch node).

Avoid D0/D1 for anything (keep for Serial/bootloader).

Recommended pinout (I^2C OLED version - simplest, no conflicts)

ELRS / control

D9 → PPM_OUT to ELRS GPIO13 (100 Ω series).

Optional future CRSF: break out a 4–pin header 3V3 / GND / UART_TX / UART_RX from any spare USART; wire later in firmware.

Joysticks (KY–023)

A0 → Right X (Roll) – wiper via 1 k, 100 nF to GND at pin

A1 → Right Y (Pitch) – wiper via 1 k, 100 nF

A2 → Left X (Yaw) – wiper via 1 k, 100 nF

A3 → Left Y (Throttle) – wiper via 1 k, 100 nF

Each stick's +V to 3V3, GND to GND. Stick push–switches are optional; you already have 4 buttons below.

Switches (momentary PTS647) - to GND, use INPUT_PULLUP

D7 → SW1 (Arm)

D8 → SW2 (Mode)

D10 → SW3 (OK/Enter)

D4 → SW4 (Back)

Indicators / actuators

D5 → LED1

D6 → LED2

D2 → Buzzer MOSFET gate (100-220 Ω series, 100 k pulldown). Buzzer + to 3V3, - to MOSFET drain. SS14 across buzzer: cathode to 3V3, anode to drain.

D3 → Haptic MOSFET gate (same gate network). ERM + to 3V3, - to drain. SS14 across ERM: cathode to 3V3, anode to drain.

VBAT measurement

A7 → VBAT divider node (27 k top to VBAT, 10 k bottom to GND), 100 nF from A7 to GND at the pin.

Display connector (your 6–pin, set to 3.3 V) - I^2C mode

Pin–1: 3V3

Pin–2: GND

Pin–3: SCL → A5

Pin–4: SDA → A4

Pin–5: NC (MISO not used on I^2C)

Pin–6: NC (CS not used on I^2C)

This version keeps SPI pins free and leaves you maximum headroom.

Alternative pinout changes if you choose SPI TFT (ST7735/ST7789)

Keep everything above unchanged except the display and two switch pins:

SPI signals (use the MCU's hardware SPI pins)

Connector Pin–3 SCK → SCK (ICSP SCK / board SCK pin)

Connector Pin–4 MOSI → MOSI (ICSP MOSI / board MOSI pin)

Connector Pin–5 MISO → MISO (if your TFT uses it; otherwise NC)

Connector Pin–6 CS → D10 (dedicated CS)

Add two control lines for SPI displays

D2 → DC (data/command)

D3 → RST

Because D2/D3 are now used for DC/RST, move the two buttons that were on D2/D3 in the I^2C plan as follows:

SW3 (OK/Enter) stays on D10 only if you are on I^2C.
For SPI, dedicate D10 to CS and move SW3 to D12 (works fine; not used if your TFT doesn't need MISO).

SW4 (Back) remains D4.

Summary (SPI):

D10 = CS (no switch here)

D2 = DC

D3 = RST

If your TFT needs MISO, keep D12 for MISO and place SW3 on another free pin (e.g., D14/D15 if available on your board).

If you want me to lock this to the exact Nano–Every Arduino pin labels for the SPI block (ICSP header vs D11-D13), say which footprint you're using and I'll pin it precisely.

Notes you'll want during bring–up

VBAT maths (27k:10k): scale = (27+10)/10 = 3.7.
Vbat = adc * (Vref/1023) * 3.7. With 8.4 V full → ≈2.27 V at A7.
Do a dummy read after mux change and average 8-16 samples.

PPM: D9 is the canonical output used by many Arduino PPM encoders; keep the wire short to the ELRS GPIO13. Configure ELRS firmware for PPM input on GPIO13.

Flyback diodes: SS14 across magnetic buzzer and ERM motor only (cathode to 3V3, anode to switch node). No diode on a piezo.

MOSFET gates: 100-220 Ω series, 100 k pulldown to GND. Keep loop area small (place diode close to load/MOSFET).

Sheet: /
File: FCSTMV1.03.kicad_sch
Title:
Size: A0 Date: **Rev:**
KiCad E.D.A. 8.0.6 Id: 1/1